lvaughn Updating README and doc strings to reflect that n_mels can now … fc5ded7 · 4 months ago

157 lines (127 loc) · 4.83 KB

Code Blame Raw

```
from functools import lru_cache
from subprocess import CalledProcessError, run
from typing import Optional, Union

import numpy as np
import torch
import torch.nn.functional as F

from .utils import exact_div

# hard-coded audio hyperparameters
SAMPLE_RATE = 16000
N_FFT = 400
HOP_LENGTH = 160
CHUNK_LENGTH = 30
N_SAMPLES = CHUNK_LENGTH * SAMPLE_RATE  # 480000 samples in a 30-second chunk
N_FRAMES = exact_div(N_SAMPLES, HOP_LENGTH)  # 3000 frames in a mel spectrogram input

N_SAMPLES_PER_TOKEN = HOP_LENGTH * 2  # the initial convolutions has stride 2
FRAMES_PER_SECOND = exact_div(SAMPLE_RATE, HOP_LENGTH)  # 10ms per audio frame
TOKENS_PER_SECOND = exact_div(SAMPLE_RATE, N_SAMPLES_PER_TOKEN)  # 20ms per audio token


def load_audio(file: str, sr: int = SAMPLE_RATE):
    """
    Open an audio file and read as mono waveform, resampling as necessary

    Parameters
```

```
        ----------
        file: str
            The audio file to open

        sr: int
            The sample rate to resample the audio if necessary

        Returns
        -------
        A NumPy array containing the audio waveform, in float32 dtype.
        """

        # This launches a subprocess to decode audio while down-mixing
        # and resampling as necessary.  Requires the ffmpeg CLI in PATH.
        # fmt: off
        cmd = [
            "ffmpeg",
            "-nostdin",
            "-threads", "0",
            "-i", file,
            "-f", "s16le",
            "-ac", "1",
            "-acodec", "pcm_s16le",
            "-ar", str(sr),
            "-"
        ]
        # fmt: on
        try:
            out = run(cmd, capture_output=True, check=True).stdout
        except CalledProcessError as e:
            raise RuntimeError(f"Failed to load audio: {e.stderr.decode()}") from e

        return np.frombuffer(out, np.int16).flatten().astype(np.float32) / 32768.0


    def pad_or_trim(array, length: int = N_SAMPLES, *, axis: int = -1):
        """
        Pad or trim the audio array to N_SAMPLES, as expected by the encoder.
        """
        if torch.is_tensor(array):
            if array.shape[axis] > length:
                array = array.index_select(
                    dim=axis, index=torch.arange(length, device=array.device)
                )

            if array.shape[axis] < length:
                pad_widths = [(0, 0)] * array.ndim
                pad_widths[axis] = (0, length - array.shape[axis])
                array = F.pad(array, [pad for sizes in pad_widths[::-1] for pad in sizes])
        else:
            if array.shape[axis] > length:
                array = array.take(indices=range(length), axis=axis)

            if array.shape[axis] < length:
```

```
                pad_widths = [(0, 0)] * array.ndim
                pad_widths[axis] = (0, length - array.shape[axis])
                array = np.pad(array, pad_widths)

        return array


    @lru_cache(maxsize=None)
    def mel_filters(device, n_mels: int) -> torch.Tensor:
        """
        load the mel filterbank matrix for projecting STFT into a Mel spectrogram.
        Allows decoupling librosa dependency; saved using:

            np.savez_compressed(
                "mel_filters.npz",
                mel_80=librosa.filters.mel(sr=16000, n_fft=400, n_mels=80),
                mel_128=librosa.filters.mel(sr=16000, n_fft=400, n_mels=128),
            )
        """
        assert n_mels in {80, 128}, f"Unsupported n_mels: {n_mels}"

        filters_path = os.path.join(os.path.dirname(__file__), "assets", "mel_filters.npz")
        with np.load(filters_path, allow_pickle=False) as f:
            return torch.from_numpy(f[f"mel_{n_mels}"]).to(device)


    def log_mel_spectrogram(
        audio: Union[str, np.ndarray, torch.Tensor],
        n_mels: int = 80,
        padding: int = 0,
        device: Optional[Union[str, torch.device]] = None,
    ):
        """
        Compute the log-Mel spectrogram of

        Parameters
        ----------
        audio: Union[str, np.ndarray, torch.Tensor], shape = (*)
            The path to audio or either a NumPy array or Tensor containing the audio waveform in 16

        n_mels: int
            The number of Mel-frequency filters, only 80 and 128 are supported

        padding: int
            Number of zero samples to pad to the right

        device: Optional[Union[str, torch.device]]
            If given, the audio tensor is moved to this device before STFT

        Returns
        -------
        torch.Tensor, shape = (n_mels, n_frames)
            A Tensor that contains the Mel spectrogram
        """
        if not torch.is_tensor(audio):
```

```
        if not torch.is_tensor(audio):
            if isinstance(audio, str):
                audio = load_audio(audio)
            audio = torch.from_numpy(audio)

        if device is not None:
            audio = audio.to(device)
        if padding > 0:
            audio = F.pad(audio, (0, padding))
        window = torch.hann_window(N_FFT).to(audio.device)
        stft = torch.stft(audio, N_FFT, HOP_LENGTH, window=window, return_complex=True)
        magnitudes = stft[..., :-1].abs() ** 2

        filters = mel_filters(audio.device, n_mels)
        mel_spec = filters @ magnitudes

        log_spec = torch.clamp(mel_spec, min=1e-10).log10()
        log_spec = torch.maximum(log_spec, log_spec.max() - 8.0)
        log_spec = (log_spec + 4.0) / 4.0
        return log_spec
```